Exit Ticket · reflect and retrieve

Exit Ticket

True or false: modular code can be easier to test independently.

Use key vocabulary from memory

1. Explain modularity in your own words.

2. Explain breakpoint in your own words.

3. Explain step into in your own words.

Confidence and next step

How confident are you? Thumbs up / side / down

Which question would you improve after checking the classroom model?

Check answers after the lesson using the teacher-guided source models.

RHS COMPUTING / OCR H446 / LESSON 043 DRAFT · TEACHER REVIEW

Do Now · retrieval

Programming Techniques · Modular Design and IDE Debugging

Name: _____ Class: _____ Date: _____

Answer four retrieval questions before reviewing earlier learning.

Q1

What does ref do in C#?

Q2

What is variable scope?

Q3

What does a void method return?

Q4

State one advantage of a hash table.

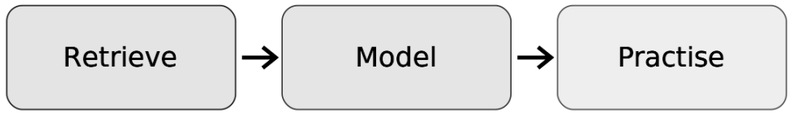
New Learning · key ideas

The important knowledge from today's lesson. Read, annotate, and explain it.

Core idea 1

Explain the important decision — Break on the array access, Step Into Sum and watch i, marks.Length and total. Diagnose the failing state before editing the loop condition.

Key visual · complete concept



Annotate the key visual

Explain which labels or features show the main process.

You Do · extended practice

Answer these selected original questions in BLACK PEN. Other exam tasks remain online.

Question 3

Apply independently
Correct: static int Add(int a,int b){ Console.WriteLine(a+b); } [2]

Check that you have used the technical terms and explained your decisions.

You Do · independent practice

Answer these selected original questions in BLACK PEN. Other exam tasks remain online.

Question 1

Apply independently

Give two benefits of modular design. [2]

Question 2

Apply independently

Describe how a breakpoint helps debug a loop. [2]

Check that you have used the technical terms and explained your decisions.

New Learning · applying the ideas

Vocabulary · use the correct technical terms

modularity — Divide code into independent components

breakpoint — Pause in the debugger

step into — Enter a method while debugging

watch — Monitor expression/variable

Check your understanding

Explain one key process in your own words using two of the terms above.

More animated examples are available in the original online lesson.

I Do · worked example

Problem

Trace a method call

Follow the worked method

1. Pause and inspect — Call Sum with [4,6]. Step Into enters the method; Step Over a call would run it without following its internal statements. Watch total=0, Length=2 and i=0.

2. Locate the first invalid access — At i=0 total becomes 4; at i=1 it becomes 10. The faulty test allows i=2, but valid indexes are 0 and 1. The next array access fails.

3. Correct and retest — Change i<=marks.Length to i<marks.Length. Retest [4,6]→10, []→0 and [7]→7. A small routine can now be tested independently and reused.

Why this method works

Identify the decision, rule or invariant that makes the method correct.

We Do · Try → Discuss → Improve

TRY independently in BLACK PEN; compare methods with a partner; improve your reasoning.

Guided problem · TRY

Discuss, then respond
A method has no return despite return type int. What is wrong?

DISCUSS

Compare how you reached the answer. Where could someone make an error?

IMPROVE

Add one precise correction or a stronger reason. Answers are not printed here.